

**Pontificia Universidad Católica Madre y Maestra
(PUCMM)**



“Práctica #2 - CRUD”

Estudiante:

Joseph Rafael Muñoz Pimentel
10155624

Samuel Olivo Pichardo
10154465

Asignatura:

Programación Web
CSTI-1910-6339

Docente:

Carlos Alfredo Camacho Guerrero

Santiago de los Caballeros, República Dominicana
16 de febrero de 2026

Introducción:

Este reporte detalla la arquitectura y funcionalidad de una aplicación web orientada al comercio electrónico, desarrollada como parte de la práctica de la asignatura Programación Web. El proyecto utiliza el lenguaje Java en conjunto con el framework Javalin, una solución para crear servicios web, y el motor de plantillas Thymeleaf para la generación de vistas dinámicas basadas en el estado del servidor.

El objetivo principal es la implementación de un sistema que permita gestionar el ciclo de vida completo de productos y usuarios (Create, Read, Update, Delete), integrando además un flujo de ventas que interactúa directamente con un inventario en tiempo real. A través de este desarrollo, se exploran conceptos como el manejo de sesiones HTTP, la seguridad mediante filtros de interceptación y la manipulación de datos financieros con precisión decimal mediante BigDecimal.

El repositorio utilizado fue el de samuel

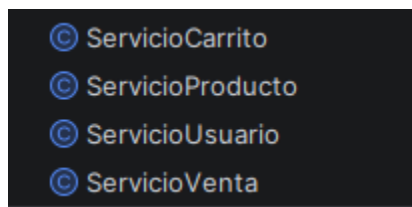
Estructura del Proyecto y Modelado de Datos

La aplicación se diseñó siguiendo un patrón de separación de responsabilidades para garantizar la escalabilidad y el mantenimiento del código. Se definieron entidades fundamentales en el paquete `org.example.models` que representan los pilares del negocio:

El proyecto se organiza básicamente en tres partes. Primero está la tabla de Usuarios, que es donde controlamos quién entra, si es admin o cliente, y si su cuenta está activa o no. Después tenemos el catálogo de productos, que centraliza todo lo del inventario: cuánto cuesta cada cosa, cuánto stock queda y si el producto está visible para la venta. Por último, está el carrito y la venta; el carrito es solo para ir guardando los productos de forma temporal mientras el usuario elige, y cuando se confirma, pasa a ser una venta permanente para que nos quede guardado todo el historial de transacciones.

Capa de Servicios y Lógica de Negocio

Para que las rutas no se volvieran un lío de lógica, lo que hicimos fue separar todo en una capa de servicios en `.services`. Ahí es donde realmente pasa la magia: los datos se guardan en `ArrayLists` en memoria y se aseguran de que las reglas del negocio se cumplan siempre igual utilizando borrado lógico. Así, si ese producto ya se había vendido antes, la venta no se rompe y el historial queda impecable.



Gestión de Sesiones y Control de Acceso

Para la seguridad, lo que hicimos fue aprovechar los atributos de sesión que ya trae Javalin. Metí unos filtros globales con `app.before` que básicamente funcionan como un portero: antes de que pase cualquier solicitud, el sistema chequea quién es el usuario y si tiene permiso.

También le agregamos lo del Referer. Esto es súper útil porque el sistema se 'acuerda' de cuál fue la última página que visitó el usuario; así, cuando se loguea o le da a 'Volver', lo manda directo a donde estaba y no lo deja perdido en el inicio. Por último, protegimos las rutas que empiezan con

/admin/**. Si alguien intenta entrar ahí y no tiene el rol de ADMIN, el filtro lo rebota de una y lo manda de regreso a la tienda para que no ande donde no debe.

```
app.before( path: "/admin/**", Context ctx -> {
    Usuario usuario = ctx.sessionAttribute(KeySession.USUARIO.name());

    if (usuario == null || usuario.getRol() != RolesUsuario.ADMIN) {
        String anteriorPath = ctx.header("Referer");
        ctx.redirect((anteriorPath != null && !anteriorPath.isEmpty()) ? anteriorPath : "/productos");
    }
});
```

Flujo de Inventario y Procesamiento de Ventas

Lo más importante que hace la práctica es el paso a paso para el carrito y la venta. Primero, el sistema se pone a revisar producto por producto del carrito y compara lo que pide el cliente contra lo que realmente hay en el ServicioProducto. Si todo está en orden y hay stock, de una vez se hace la actualización restando esas unidades del inventario global para que nadie más las agarre. Finalmente, se genera el objeto Venta. Ahí guardamos la fecha, quién compró y, se guarda el precio ahí. O sea que registramos el precio que tenía el producto justo en ese momento por si después sube o baja, para que el historial no se mueva.

```
app.get( path: "/carrito/agregar/{id}", Context ctx -> {
    int idProducto = Integer.parseInt(ctx.pathParam(s: "id"));
    Producto producto = servicioProducto.buscarActivoPorId(idProducto);
    Usuario usuario = ctx.sessionAttribute(KeySession.USUARIO.name());

    if (producto != null && producto.getCantidad() > 0) {
        if (servicioCarrito.buscarPorId(usuario.getId()) == null) {
            servicioCarrito.crear(usuario);
        }

        servicioCarrito.agregarProducto(usuario.getId(), producto);
        ctx.redirect( location: "/productos");
    } else {
        ctx.result( resultString: "Lo sentimos, no hay stock suficiente de este producto.");
    }
});
```

Interfaz de Usuario con Thymeleaf

Las plantillas HTML utilizan fragmentos de código para renderizar tablas de productos que permiten la selección por filas mediante JavaScript. Los administradores visualizan botones adicionales de "Editar" y "Eliminar" que se activan dinámicamente al seleccionar un elemento de la tabla.

Conclusión:

La realización de esta asignación nos permitió implementar la validación en el lado del servidor. La experiencia de integrar un sistema de sesiones fue interesante, ya que permite transformar una aplicación estática en una plataforma multiusuario segura. El mayor aprendizaje fue en la gestión de la consistencia de datos: asegurar que el stock se descuente correctamente, que el historial de ventas sea inmutable o algo tan sencillo como la asociación entre usuario y carrito.

Referencias:

Documentation - Javalin - A lightweight Java and Kotlin web framework. Create REST APIs in Java or Kotlin easily. Javalin. <https://javalin.io/documentation#getting-started>

Tutorial: Using Thymeleaf.

<https://www.thymeleaf.org/doc/tutorials/3.1/usingthymeleaf.html#using-thtext-and-externalizing-text>

Repositorio:

<https://github.com/samuelolivo/st-icc-352-002-sxop0001/tree/main/practica-2>